

=====

BIBLE LINUX – §15 – DÉVELOPPEMENT & COMPILATION (COMPLÉMENTS)

LÉGENDE : [SUDO] = droits root requis | [USER] = utilisateur normal

NOTE : Voir aussi §10 pour git, make, gcc/g++, gdb, valgrind, perf, ar, ld, as, nm, strip, objdump, readelf, addr2line, ldd, ctags, python3, pip, node, npm, cutter-re/rizin.

=====

=====

A. OUTILS BINUTILS & ANALYSE ELF (COMPLÉMENTS)

=====

[USER] objcopy

DESCRIPTION : Copie et transforme des fichiers objets ELF – extrait des sections, convertit des formats, ajoute ou supprime des segments.

POURQUOI : Manipuler des binaires : extraire le .text, convertir en raw binaire, séparer les symboles de débogage, créer des fichiers de firmware.

CONTEXTE : Embarqué, packaging, débogage avec symboles séparés, reverse engineering.

OPTIONS :

-O FORMAT Format de sortie (elf32-i386, elf64-x86-64, binary, ihex, srec, verilog, symbolsrec...)

-I FORMAT Format d'entrée (auto-déTECTÉ par défaut)

-j SECTION Copie seulement cette section

-R SECTION Supprime cette section

--strip-debug Supprime les infos de débogage

--strip-unneeded Supprime les symboles inutiles au linkage

--strip-all Supprime tous les symboles

--only-keep-debug Garde seulement les infos de débogage

--add-gnu-debuglink=FICHIER Ajoute un lien vers un fichier de debug séparé

--set-section-flags SECT=FLAGS Modifie les flags d'une section

--add-section NOM=FICHIER Ajoute une section depuis un fichier

--rename-section ANCIEN=NOUVEAU Renomme une section

--change-start INCR Modifie l'adresse de départ

--change-section-address SECT=INCR Change l'adresse d'une section

--globalize-symbol SYM Rend un symbole global

--localize-symbol SYM Rend un symbole local

--weaken-symbol SYM Affaiblit un symbole

-B ARCH Architecture cible (pour cross-compilation)

--file-alignment N Alignement des sections dans le fichier

--section-alignment N Alignement des sections en mémoire

-v Verbeux

EXEMPLES :

Séparer les symboles de débogage (package -debuginfo) :

\$ objcopy --only-keep-debug programme programme.debug

\$ objcopy --strip-debug programme programme.stripped

\$ objcopy --add-gnu-debuglink=programme.debug programme.stripped

Convertir en binaire brut (firmware) :

\$ objcopy -O binary -j .text programme code.bin

\$ objcopy -O ihex programme firmware.hex # Format Intel HEX

\$ objcopy -O srec programme firmware.srec # Format Motorola S-record

Embarquer un fichier dans un objet (ex: données, clé) :

\$ objcopy -I binary -O elf64-x86-64 -B i386:x86-64 \

--rename-section .data=.rodata,alloc,load,readonly,data,contents \

data.bin data_embedded.o

Puis linker avec le programme : les symboles _binary_data_bin_start/_end sont accessibles

Ajouter une section personnalisée :

\$ objcopy --add-section .config=config.dat programme programme_with_config

[USER] size

DESCRIPTION : Affiche la taille des sections d'un fichier objet ou exécutable (.text, .data, .bss) et la taille totale.

POURQUOI : Analyser la répartition mémoire d'un binaire, optimiser la taille du code (essentiel en embarqué), comparer avant/après optimisation.

CONTEXTE : Développement embarqué, optimisation, analyse de binaires.

OPTIONS :

-A / --sysv Format SysV (section par section)

-B / --berkeley Format Berkeley (défaut : text+data+bss)

-G / --gnu Format GNU (similaire à Berkeley mais plus détaillé)

```

-d / --radix=10      Affichage décimal
-o / --radix=8       Affichage octal
-x / --radix=16      Affichage hexadécimal
-t / --totals        Affiche les totaux (pour plusieurs fichiers)
--common             Inclut les symboles communs dans .bss
-f / --format=FORMAT Format de sortie (berkeley, sysv, gnu)
FICHIER [...]       Un ou plusieurs fichiers à analyser
SECTIONS            :
.text               Code exécutable (instructions)
.data               Données initialisées (variables globales avec valeur)
.bss                Données non initialisées (zéro au démarrage, n'occupe pas le fichier)
dec                 Taille totale en décimal
hex                 Taille totale en hexadécimal
EXEMPLES            :
$ size mon_programme      # Taille des sections
$ size -A mon_programme   # Format SysV (toutes les sections)
$ size *.o                # Analyse chaque fichier objet
$ size -t *.o              # Avec totaux
$ size -x -A mon_programme # Hex, toutes sections

```

```
[USER] strings
```

```

DESCRIPTION : Extrait les chaînes de caractères imprimables d'un fichier binaire
              (toute séquence d'au moins N caractères ASCII/Unicode imprimables).
POURQUOI    : Trouver des informations dans un binaire sans le désassembler :
              messages d'erreur, chemins de fichiers, URLs, clés API hardcodées,
              versions, noms de fonctions, indices de débogage.
CONTEXTE    : Reverse engineering, sécurité, analyse de malware, débogage.
OPTIONS      :
-a / --all           Scanne tout le fichier (pas seulement les sections data)
-f / --print-file-name Préfixe chaque chaîne du nom de fichier
-n N / --bytes=N     Longueur minimale (défaut: 4)
-o                  Affiche le décalage (octal)
-t FORMAT            Affiche le décalage (d=décimal, o=octal, x=hex)
-e ENCODAGE          Encodage (s=7-bit, S=8-bit, b=16-bit big-endian,
                      l=16-bit little-endian, B=32-bit BE, L=32-bit LE)
-U METHOD            Méthode Unicode (default, locale, escape, highlight,
                      highlight-default, highlight-locale)
-w                  Inclut les espaces comme caractère de chaîne
-s / --output-separator=SEP Séparateur de sortie
FICHIER [...]       Fichier(s) à analyser
EXEMPLES          :
$ strings /usr/bin/ls      # Strings dans ls
$ strings -n 8 malware.bin # Strings de 8+ caractères
$ strings -a firmware.bin  # Tout le fichier (pas seulement data)
$ strings -t x fichier.bin  # Avec offset hexadécimal
$ strings executable | grep -i "pass\\|key\\|secret\\|token\\|http" # Chercher des secrets
$ strings -e l fichier.dll  # Strings UTF-16 (DLL Windows)
$ strings programme | grep "^/" # Chemins absolus hardcodés

```

```
[USER] c++filt
```

```

DESCRIPTION : Démangleuse de noms C++ – convertit les noms "mangled" internes
              du compilateur (ex: _ZN3FooC1Ei) en noms lisibles (Foo::Foo(int)).
POURQUOI    : Les compilateurs C++ "manglent" les noms de fonctions pour encoder
              les surcharges, espaces de noms et paramètres. c++filt les traduit
              en noms humainement lisibles pour déboguer des erreurs de linkage.
CONTEXTE    : Développement C++, débogage d'erreurs de linkage, analyse de binaires.
OPTIONS      :
-_ / --strip-underscore Supprime le _ initial (style ancien compilateur)
-n / --no-strip-underscore Ne supprime pas le _ initial
-p / --no-params         N'affiche pas les types de paramètres
-t / --types             Démangle aussi les noms de types
-i / --no-verbose        Pas de noms de fonctions dans les erreurs
-s FORMAT               Format de mangling (auto, gnu, lucid, arm, hp,
                        edg, gnu-v3, java, gnat)
[NOM ...]              Noms à démangler (stdin si absent)
EXEMPLES          :
$ c++filt _ZN3FooC1Ei      # → Foo::Foo(int)
$ c++filt _ZNSt6vectorIiSaIiEEC1Ev # → std::vector<int, std::allocator<int> >::vector()
$ nm mon_prog | c++filt    # Démangler tous les symboles de nm
$ nm -C mon_prog           # Équivalent (nm avec démangement intégré)
$ c++filt < erreur_linker.txt # Démangler une sortie d'erreur

```

```
# Depuis stdin interactif :
$ c++filt
_ZN3FooC1Ei
Foo::Foo(int)
```

```
[USER] elfedit
```

DESCRIPTION : Modifie directement les champs de l'en-tête ELF d'un binaire (type, architecture, OS ABI, interpréteur dynamique, flags...).

POURQUOI : Ajuster des métadonnées ELF sans recompiler : changer l'OS ABI, corriger l'architecture cible, modifier les flags ELF.

CONTEXTE : Cross-compilation, portage, correction de binaires, embarqué.

OPTIONS :

- input-mach=TYPE Filtre par architecture machine en entrée
- output-mach=TYPE Modifie l'architecture machine
- input-type=TYPE Filtre par type ELF en entrée (ET_EXEC, ET_DYN...)
- output-type=TYPE Modifie le type ELF
- input-osabi=OSABI Filtre par OS ABI en entrée
- output-osabi=OSABI Modifie l'OS ABI (NONE, HPUX, NETBSD, LINUX, SOLARIS, FREEBSD, ARM, STANDALONE...)
- enable-x86-feature=FEATURE Active une feature x86 (ibt, shstk)
- disable-x86-feature=FEATURE Désactive une feature x86
- v / --verbose Verbeux
- FICHER [...] Fichier(s) à modifier (en place !)

EXEMPLES :

```
$ elfedit --output-osabi LINUX mon_programme # Change l'ABI vers Linux
$ elfedit --output-type ET_DYN mon_prog      # Transforme un exécutable en lib
$ elfedit --output-mach EM_X86_64 fichier.o  # Change l'architecture cible
```

```
[USER] ranlib
```

DESCRIPTION : Génère (ou met à jour) l'index d'une bibliothèque statique (.a) pour accélérer le linkage.

POURQUOI : Les bibliothèques .a sont des archives de fichiers .o. ranlib ajoute un index des symboles qui permet à ld de trouver rapidement les fonctions sans scanner toute l'archive. Requis après ar ou sur certains systèmes anciens.

CONTEXTE : Compilation de bibliothèques statiques, cross-compilation.

OPTIONS :

- t Met à jour le timestamp seulement (sans recalculer)
- v Verbeux
- D Déterministe (timestamps à zéro pour reproductibilité)
- U Non-déterministe (timestamps réels)
- FICHER [...] Bibliothèque(s) .a à indexer

EXEMPLES :

```
$ ar rcs libmachin.a *.o # Crée l'archive (r=insert, c=create, s=index)
$ ranlib libmachin.a     # Crée/met à jour l'index (équivalent à ar s)
$ ranlib -D libmachin.a  # Index déterministe (builds reproductibles)
# Note : ar rcs inclut déjà s (index), ranlib est souvent redondant
# mais nécessaire pour la compatibilité avec les Makefiles classiques
```

B. ELFUTILS (SUITE EU-*)

```
[USER] eu-readelf
```

DESCRIPTION : Version elfutils de readelf – affiche les informations ELF avec support debuginfod et meilleure gestion des fichiers DWARF.

POURQUOI : Comme readelf (binutils) mais avec support des extensions DWARF5, debuginfod et quelques fonctions supplémentaires.

CONTEXTE : Analyse de binaires, débogage, développement bas niveau.

OPTIONS CLÉS :

- a / --all Toutes les informations
- h / --file-header En-tête ELF
- S / --section-headers En-têtes des sections
- l / --program-headers En-têtes des segments (programme)
- d / --dynamic Section dynamique
- s / --syms / --symbols Symboles
- dyn-syms Symboles dynamiques
- r / --relocs Relocations
- n / --notes Notes (build-id, ABI...)

```

-e / --headers          Tous les headers
-x / --hex-dump=SECTION Dump hexadécimal d'une section
-p / --string-dump=SECTION Dump chaînes d'une section
-w / --debug-dump[=QUOI] Infos DWARF (info, abbrev, line, loc, ranges,
                        pubnames, aranges, macro, frames, str, frames-interp)
--dwarf-depth=N         Profondeur des DIE DWARF
--dwarf-start=N          Commence à ce DIE
-W / --wide             Format large (pas de coupure)

```

EXEMPLES :

```

$ eu-readelf -h /usr/bin/bash          # En-tête ELF
$ eu-readelf -s /usr/lib/libm.so       # Symboles
$ eu-readelf -d /usr/lib/libssl.so     # Dépendances dynamiques
$ eu-readelf --debug-dump=info prog    # Infos DWARF complètes
$ eu-readelf -n mon_prog               # Build-ID et notes

```

```
[USER] eu-objdump
```

DESCRIPTION : Version elfutils de objdump – désassembleur et analyseur de fichiers objets avec support DWARF avancé.

POURQUOI : Désassembler des binaires ELF avec accès aux informations de débogage DWARF intégrées pour un contexte source/assembleur.

CONTEXTE : Analyse de binaires, débogage bas niveau.

OPTIONS CLÉS :

```

-d / --disassemble      Désassemble les sections exécutables
-D / --disassemble-all Désassemble toutes les sections
-S / --source           Intercala le code source (si info DWARF)
-s / --full-contents    Contenu complet des sections
-T / --dynamic-syms     Symboles dynamiques
--dwarf[=QUOI]          Infos DWARF
-j SECTION              Limite à cette section
-l / --line-numbers     Numéros de lignes source
-M OPTS                 Options désassembleur (intel, att)

```

EXEMPLES :

```

$ eu-objdump -d /usr/bin/ls          # Désassemble ls
$ eu-objdump -S mon_programme        # Avec code source intercalé
$ eu-objdump -d -M intel programme  # Syntaxe Intel

```

```
[USER] eu-nm
```

DESCRIPTION : Version elfutils de nm – liste les symboles d'un fichier objet avec support debuginfod.

POURQUOI : Lister les symboles avec résolution automatique depuis debuginfod pour les binaires système sans paquet -debuginfo installé.

OPTIONS CLÉS :

```

-a / --debug-syms       Symboles de débogage
-D / --dynamic           Symboles dynamiques
-g / --extern-only       Symboles globaux seulement
-u / --undefined-only    Symboles non définis
-C / --demangle          Démangling C++
--defined-only           Symboles définis
-n / --numeric-sort      Tri numérique
-p / --no-sort           Pas de tri
-S / --print-size        Affiche la taille des symboles

```

EXEMPLES :

```

$ eu-nm /usr/lib/libm.so          # Symboles de libm
$ eu-nm -D -C /usr/lib/libstdc++.so # Symboles dyn. C++ démantés
$ eu-nm -u mon_programme          # Symboles importés

```

```
[USER] eu-strip
```

DESCRIPTION : Version elfutils de strip – supprime les symboles et informations de débogage d'un binaire ELF pour réduire sa taille.

POURQUOI : Réduire la taille des binaires pour la distribution, tout en optionnellement préservant les infos de débogage dans un fichier séparé.

OPTIONS CLÉS :

```

-f FICHIER / --output-file=FICHIER Fichier de sortie (sinon modifie en place)
-F FICHIER / --remove-comment       Supprime les commentaires
-s / --strip-all                   Supprime tout
-d / --strip-debug                  Supprime seulement les infos de débogage
-g                                  Alias de --strip-debug
--strip-sections                    Supprime aussi les en-têtes de sections
-R / --remove-section=SECTION       Supprime une section spécifique

```

```

-k / --keep-symbol=SYM      Garde ce symbole
--remove-comment            Supprime la section commentaire
-p / --preserve-dates       Préserve les timestamps
EXEMPLES :
$ eu-strip programme        # Strip en place
$ eu-strip -f programme.debug --strip-debug programme # Sépare les debug info
$ eu-strip -o programme.min programme # Vers un nouveau fichier

```

```
[USER] eu-ar
```

```

DESCRIPTION : Version elfutils de ar – crée et manipule des archives de fichiers
              objets (.a, bibliothèques statiques).
POURQUOI    : Créer des bibliothèques statiques avec support ELF étendu.
SYNTAXE     : eu-ar OPÉRATION[MODIF] ARCHIVE [MEMBRES...]
OPÉRATIONS  :
  r Insère ou remplace les membres
  x Extrait les membres
  t Liste les membres
  d Supprime les membres
  q Ajoute (quick, sans vérification)
  s Crée/met à jour l'index des symboles
  m Déplace les membres
MODIFICATEURS :
  c Crée l'archive sans avertissement si absente
  s Crée l'index des symboles (armap)
  v Verbeux
  u Met à jour seulement si plus récent
  D Déterministe (timestamps à zéro)
EXEMPLES    :
$ eu-ar rcs libmonlib.a objet1.o objet2.o # Crée une bibliothèque statique
$ eu-ar t libmonlib.a                     # Liste le contenu
$ eu-ar x libmonlib.a                     # Extrait tous les objets

```

```
[USER] eu-make-debug-archive
```

```

DESCRIPTION : Crée une archive de symboles de débogage pour une bibliothèque
              ou un ensemble de fichiers objets.
POURQUOI    : Générer des archives .debug pour les paquets -debuginfo,
              séparant les informations de débogage des binaires de production.
CONTEXTE     : Packaging, création de paquets -debuginfo, infrastructure CI/CD.
EXEMPLES     :
$ eu-make-debug-archive -o lib.debug lib.so # Archive debug de lib.so

```

```
[USER] eu-stacktrace
```

```

DESCRIPTION : Lit les données de déroulement de pile (stack unwinding) d'un
              processus ou core dump et génère une trace lisible.
POURQUOI    : Obtenir un backtrace symbolisé d'un crash sans GDB interactif,
              avec résolution automatique via debuginfod.
CONTEXTE     : Débogage automatisé, CI/CD, analyse de crashes en production.
EXEMPLES     :
$ eu-stacktrace -p 1234          # Trace du PID 1234
$ eu-stacktrace -e programme core # Depuis un core dump

```

C. BIBLIOTHÈQUES SYSTÈME & LINKAGE DYNAMIQUE

```
[SUDO] ldconfig
```

```

DESCRIPTION : Configure les liens et le cache du chargeur dynamique (ld.so).
              Met à jour /etc/ld.so.cache après l'installation de bibliothèques.
POURQUOI    : Quand on installe une bibliothèque .so manuellement, le système ne
              la voit pas jusqu'à ce que ldconfig soit exécuté. Résout les erreurs
              "error while loading shared libraries: libxxx.so: cannot open".
CONTEXTE     : Installation manuelle de bibliothèques, développement, packaging.
OPTIONS     :
  -v / --verbose      Affiche les bibliothèques traitées
  -n RÉPERTOIRE       Traite seulement ce répertoire (ne met pas à jour le cache)
  -N                  Ne met pas à jour le cache (scan seulement)
  -X                  Ne met pas à jour les liens symboliques

```

```

-l BIBLIOTHÈQUE      Mode manuel : traite ce fichier .so
-p / --print-cache   Affiche les bibliothèques dans le cache actuel
-C CACHE             Utilise ce fichier cache alternatif
-f CONF              Utilise ce fichier de config alternatif
-r RÉPERTOIRE        Utilise ce répertoire comme racine
RÉPERTOIRE [...]     Répertoires supplémentaires à scanner
FICHIERS DE CONFIG :
/etc/ld.so.conf       Config principale (liste les répertoires)
/etc/ld.so.conf.d/*.conf  Configs supplémentaires (un par paquet)
/etc/ld.so.cache      Cache binaire généré par ldconfig
EXEMPLES :
$ ldconfig             # Met à jour le cache (après install)
$ ldconfig -v          # Met à jour en affichant les libs
$ ldconfig -p          # Liste toutes les libs connues du système
$ ldconfig -p | grep libssl # Trouver libssl dans le cache
$ ldconfig -n /usr/local/lib # Scanne un répertoire (sans cache global)
# Après installation d'une lib dans /usr/local/lib :
$ echo "/usr/local/lib" > /etc/ld.so.conf.d/local.conf && ldconfig
# Utilisation directe (sans ldconfig) :
$ LD_LIBRARY_PATH=/chemin/vers/lib mon_programme

```

```
[USER] getconf
```

```

DESCRIPTION : Affiche les valeurs des paramètres de configuration du système
              (limites POSIX, tailles de types, capacités du système de fichiers).
POURQUOI    : Connaître les limites système réelles (taille max d'un chemin,
              nombre max d'arguments, taille d'une page mémoire) pour écrire
              du code portable et conforme POSIX.
CONTEXTE    : Développement portable, scripts shell, audit de configuration.
OPTIONS     :
  -a [CHEMIN] Affiche toutes les variables (pour ce chemin si donné)
  -v SPEC     Spécification (ex: POSIX_V7_LP64_OFF64)
  VAR         Nom de la variable à lire
  VAR CHEMIN  Variable dépendante d'un chemin (ex: PATH_MAX /)
VARIABLES UTILES :
  PAGE_SIZE / PAGESIZE Taille d'une page mémoire (ex: 4096)
  ARG_MAX           Taille max des arguments de commande
  PATH_MAX          Longueur max d'un chemin (ex: 4096)
  NAME_MAX          Longueur max d'un nom de fichier
  OPEN_MAX          Nombre max de fichiers ouverts simultanément
  NGROUPS_MAX       Nombre max de groupes
  LONG_BIT          Taille d'un long en bits (32 ou 64)
  WORD_BIT          Taille d'un int en bits
  INT_MAX / LONG_MAX Valeurs maximales des types entiers
  HOST_NAME_MAX     Longueur max d'un nom d'hôte
  LOGIN_NAME_MAX    Longueur max d'un nom de login
  _CS_GNU_LIBC_VERSION Version de la glibc
  _CS_GNU_LIBPTHREAD_VERSION Version de libpthread
EXEMPLES :
$ getconf PAGE_SIZE      # Taille de page (typiquement 4096)
$ getconf ARG_MAX        # Limite des arguments
$ getconf PATH_MAX /     # Longueur max d'un chemin
$ getconf LONG_BIT       # Architecture 32 ou 64 bits
$ getconf _CS_GNU_LIBC_VERSION # Version glibc (ex: glibc 2.38)
$ getconf -a | grep -i "max\|size" # Toutes les limites
$ getconf -a /proc       # Limites pour /proc

```

```
[USER] dltest
```

```

DESCRIPTION : Teste le chargement dynamique d'une bibliothèque partagée via dlopen().
              Vérifie qu'une .so peut être chargée et qu'un symbole y est accessible.
POURQUOI    : Diagnostiquer des problèmes de chargement dynamique (dépendances
              manquantes, symboles introuvables, incompatibilité ABI) avant
              de les reproduire dans une application.
CONTEXTE    : Développement de bibliothèques, débogage de plugins, portage.
EXEMPLES    :
$ dltest libmachin.so    # Tente de charger la bibliothèque
$ dltest libmachin.so ma_fonction # Vérifie que le symbole est accessible
$ dltest /usr/lib64/libssl.so.3 # Test libssl

```

```
[USER] mcheck
```

DESCRIPTION : Outil de vérification de la cohérence du tas (heap) pour détecter les corruptions mémoire dans les programmes C utilisant malloc/free.

POURQUOI : Détecter les double-free, les corruptions du tas, les écritures hors limites sur des blocs malloc avant qu'ils ne causent des crashes.

CONTEXTE : Débogage mémoire, développement C, alternative légère à valgrind.

NOTE : mcheck est une fonction de la glibc activée via LD_PRELOAD ou l'utilitaire en ligne de commande. Se combine avec MALLOC_CHECK_.

EXEMPLES :

```
# Via variable d'environnement (plus courant) :
$ MALLOC_CHECK_=3 mon_programme      # 3=abort immédiat sur corruption
$ MALLOC_CHECK_=2 mon_programme      # 2=message d'erreur + abort
$ MALLOC_CHECK_=1 mon_programme      # 1=message d'erreur seulement

# Via LD_PRELOAD :
$ LD_PRELOAD=libmcheck.so mon_programme

# Appel direct dans le code (mcheck.h) :
mcheck(NULL);                        # Active la vérification
mcheck_check_all();                  # Vérifie tout le tas maintenant
```

```
[USER] debuginfod-find
```

DESCRIPTION : Recherche et téléchargement des fichiers de débogage (debuginfo), exécutables sources et sections depuis un serveur debuginfod.

POURQUOI : Obtenir les symboles de débogage pour des binaires système sans installer des gigaoctets de paquets *-debuginfo. Debuginfod les télécharge à la demande depuis un serveur centralisé.

CONTEXTE : Débogage de binaires système, analyse de core dumps, développement.

SOUS-COMMANDES :

```
debuginfo BUILD-ID      Télécharge le fichier debuginfo pour ce build-id
executable BUILD-ID     Télécharge l'exécutable pour ce build-id
source BUILD-ID /chemin/source  Télécharge le fichier source
section BUILD-ID SECTION  Télécharge une section ELF spécifique
```

OPTIONS :

```
-v      Verbeux
-n      Simulation (ne télécharge pas)
-D / --debug-cache-dir=RÉPERTOIRE  Répertoire de cache alternatif
```

VARIABLES D'ENVIRONNEMENT :

```
DEBUGINFOD_URLS      Serveurs debuginfod (séparés par espace)
                     Fedora: https://debuginfod.fedoraproject.org/
                     Ubuntu: https://debuginfod.ubuntu.com/
                     Debian: https://debuginfod.debian.net/
```

EXEMPLES :

```
# Trouver le build-id d'un binaire :
$ eu-readelf -n /usr/bin/bash | grep "Build ID"

# Télécharger les debuginfo pour bash :
$ DEBUGINFOD_URLS="https://debuginfod.fedoraproject.org/" \
  debuginfod-find debuginfo $(eu-readelf -n /usr/bin/bash | awk '/Build ID/{print $NF}')

# GDB avec debuginfod automatique :
$ DEBUGINFOD_URLS="https://debuginfod.fedoraproject.org/" gdb /usr/bin/bash core
```

D. BIBLIOTHÈQUES & DÉPENDANCES

```
[USER] pkg-config / pkgconf
```

DESCRIPTION : Récupère les options de compilation et de linkage pour les bibliothèques installées (chemins d'en-têtes, flags, bibliothèques à lier).

POURQUOI : Éviter de coder en dur les chemins de bibliothèques dans les Makefiles. pkg-config/pkgconf interrogent les fichiers .pc des bibliothèques pour fournir automatiquement les bons flags de compilation.

CONTEXTE : Compilation de logiciels, écriture de Makefiles et scripts de build.

OPTIONS :

```
--cflags NOM      Flags de compilation (-I/chemin/include...)
--libs NOM        Flags de linkage (-L/chemin -lnom...)
--libs-only-L NOM  Seulement les -L
--libs-only-l NOM  Seulement les -l
--libs-only-other NOM  Autres flags de linkage
--cflags-only-I NOM  Seulement les -I
--cflags-only-other NOM  Autres flags de compilation
--modversion NOM   Version de la bibliothèque
--exists NOM       Code de retour 0 si la lib est disponible
--atleast-version=VERSION NOM  Vérifie la version minimale
--exact-version=VERSION NOM  Vérifie la version exacte
```

```

--max-version=VERSION NOM    Vérifie la version maximale
--list-all                  Liste toutes les bibliothèques disponibles
--print-variables NOM        Liste les variables d'un .pc
--variable=NOM NOM_PKG      Valeur d'une variable du .pc
--static                     Flags pour linkage statique
--msvc-syntax                Format MSVC (Windows)
--validate NOM              Valide le fichier .pc
NOM [NOM2...]               Un ou plusieurs noms de bibliothèques
FICHIERS .PC :
/usr/lib/pkgconfig/          Emplacement standard
/usr/lib64/pkgconfig/        Lib 64-bit
/usr/share/pkgconfig/        Bibliothèques indépendantes de l'architecture
PKG_CONFIG_PATH              Variable d'env pour chemins supplémentaires
EXEMPLES :
$ pkg-config --cflags --libs gtk4      # Flags pour compiler avec GTK4
$ pkg-config --modversion openssl      # Version d'OpenSSL installée
$ pkg-config --exists libpng && echo "libpng disponible"
$ pkg-config --atleast-version=1.2 openssl || echo "OpenSSL trop vieux"
$ pkg-config --list-all | grep -i "ssl\|crypto\|tls" # Chercher des libs SSL
$ pkg-config --list-all | wc -l        # Nombre de bibliothèques disponibles
# Dans un Makefile :
#   CFLAGS += $(shell pkg-config --cflags gtk4)
#   LDFLAGS += $(shell pkg-config --libs gtk4)
# Dans CMakeLists.txt :
#   find_package(PkgConfig REQUIRED)
#   pkg_check_modules(GTK4 REQUIRED gtk4)
$ pkg-config --variable=prefix openssl # Répertoire d'installation d'OpenSSL
$ PKG_CONFIG_PATH=/usr/local/lib/pkgconfig pkg-config --libs malib # Lib perso

```

E. LANGAGES DE SCRIPT

[USER] perl

```

DESCRIPTION : Langage de scripting polyvalent – puissant pour le traitement de
               texte, les expressions régulières, l'administration système et le web.
POURQUOI    : Omniprésent sur les systèmes Unix/Linux (souvent requis par les outils
               système). Excellents regex, manipulation de texte et de fichiers.
CONTEXTE    : Scripts système, traitement de données, administration, glue code.
OPTIONS     :
-e 'CODE'    Exécute le code directement (one-liner)
-n           Boucle sur stdin ligne par ligne (sans print auto)
-p           Boucle sur stdin avec print automatique
-i[SUFFIX]   Édition en place des fichiers (-i.bak = backup)
-l           Gère automatiquement les fins de ligne
-a           Split automatique en tableau @F (avec -n ou -p)
-F SÉPARATEUR Séparateur pour -a (ex: -F:)
-w           Active les avertissements
-W           Active tous les avertissements
-X           Désactive les avertissements
-d           Mode débogage (perl debugger)
-c           Vérifie la syntaxe sans exécuter
-T           Mode taint (sécurité)
-M MODULE    Charge un module au démarrage
-v           Version de Perl
-V           Infos de compilation
FICHIER      Script Perl à exécuter
EXEMPLES    :
# One-liners (les plus utiles) :
$ perl -e 'print "Hello World\n"'
$ perl -p -i.bak -e 's/ancien/nouveau/g' fichier.txt # Remplace en place
$ perl -p -i -e 's/\r\n/\n/g' fichier.txt             # Convertit CRLF→LF
$ perl -n -e 'print if /motif/' fichier.txt           # Filtre (comme grep)
$ perl -ane 'print $F[2], "\n"' fichier.txt           # 3e champ (comme awk)
$ perl -F: -ane 'print $F[0], "\n"' /etc/passwd       # 1er champ de /etc/passwd
$ perl -e 'print localtime . "\n"'                   # Date formatée
$ perl -MPOSIX -e 'print floor(3.7), "\n"'            # Module POSIX
$ perl -0777 -p -e 's/\n/ /gs' fichier.txt           # Jointure de lignes
# Vérification de syntaxe :
$ perl -c mon_script.pl
# Lancer un script :
$ perl mon_script.pl
$ perl -w mon_script.pl                               # Avec avertissements

```



```
[USER] python3 / python (complément – voir §10 pour la base)
```

DESCRIPTION : Interpréteur Python 3 – langage généraliste de haut niveau.

OPTIONS SUPPLÉMENTAIRES :

-c 'CODE'	Exécute le code en one-liner
-m MODULE	Exécute un module comme script (python3 -m http.server)
-i	Mode interactif après exécution du script
-u	Stdout/stderr non bufferisés
-v	Mode verbeux (imports tracés)
-W ACTION	Filtres d'avertissements (ignore, default, error, always)
-O	Optimisation de base (supprime assert)
-OO	Optimisation avancée (supprime assert + docstrings)
-B	Ne crée pas de fichiers .pyc
-S	Pas d'import automatique de site.py
-x	Ignore la première ligne (shebang non-standard)
-X OPTION	Options spéciales (dev, utf8, faulthandler...)
--check-hash-based-pycs MODE	Vérifie les .pyc basés sur hash

ONE-LINERS UTILES :

```
$ python3 -c "import sys; print(sys.version)"
$ python3 -c "import json,sys; print(json.dumps(json.load(sys.stdin), indent=2))" # Format JSON
$ python3 -m json.tool fichier.json # Valide et formate JSON
$ python3 -m http.server 8000 # Serveur HTTP simple dans le répertoire courant
$ python3 -m http.server 8000 --directory /var/www # Serveur HTTP sur un répertoire
$ python3 -m venv .venv # Crée un environnement virtuel
$ python3 -c "import hashlib; print(hashlib.md5(b'texte').hexdigest())"
$ python3 -c "from datetime import datetime; print(datetime.now().isoformat())"
$ python3 -m timeit "'-'.join(str(n) for n in range(100))" # Benchmark
$ python3 -m cProfile mon_script.py # Profiling
$ python3 -m pdb mon_script.py # Débogueur interactif
$ python3 -m py_compile mon_script.py # Vérification syntaxe
```

```
[USER] python-config / python3-config
```

DESCRIPTION : Fournit les flags de compilation et de linkage nécessaires pour intégrer l'interpréteur Python dans un programme C/C++.

POURQUOI : Pour embarquer Python dans une application C (via l'API Python/C), les flags sont complexes et dépendent de l'installation. python-config les fournit automatiquement (comme pkg-config).

CONTEXTE : Développement d'extensions Python en C, embarquement de Python.

OPTIONS :

--cflags	Flags de compilation (-I...)
--ldflags	Flags de linkage (-L... -lpython3.x...)
--libs	Bibliothèques seulement (-lpython...)
--includes	Répertoires d'en-têtes seulement
--prefix	Répertoire d'installation de Python
--exec-prefix	Répertoire exécutable
--extension-suffix	Suffixe des extensions (.cpython-312-x86_64-linux-gnu.so)
--abiflags	Flags ABI
--configdir	Répertoire de configuration
--embed	Pour l'embarquement (pas juste extension)

EXEMPLES :

```
$ python3-config --cflags --ldflags # Tous les flags nécessaires
$ python3-config --extension-suffix # Suffixe pour les extensions .so
# Dans un Makefile pour une extension Python :
# CFLAGS += $(shell python3-config --cflags)
# LDFLAGS += $(shell python3-config --ldflags)
```

```
[USER] pydoc / pydoc3
```

DESCRIPTION : Affiche la documentation des modules, fonctions et classes Python directement dans le terminal ou via un serveur web local.

POURQUOI : Consulter la documentation Python offline sans navigateur, directement depuis la ligne de commande pendant le développement.

CONTEXTE : Développement Python, consultation de documentation offline.

OPTIONS/USAGES :

pydoc MODULE	Affiche la doc du module dans le pager
pydoc -k MOT_CLÉ	Recherche par mot-clé dans les titres
pydoc -p PORT	Lance un serveur web sur ce port
pydoc -b	Lance serveur + ouvre le navigateur
pydoc -n HOSTNAME	Hostname du serveur (défaut: localhost)
pydoc -w MODULE	Génère un fichier HTML

```

EXEMPLES      :
$ pydoc os          # Documentation du module os
$ pydoc os.path.join # Doc d'une fonction spécifique
$ pydoc3 json.JSONDecoder # Doc d'une classe
$ pydoc -k "sort"    # Tous les modules avec "sort"
$ pydoc -b           # Serveur web + navigateur
$ pydoc -p 9000       # Serveur web sur port 9000
$ pydoc -w os > /dev/null # Génère os.html dans le répertoire courant

```

F. NODE.JS & JAVASCRIPT (COMPLÉMENTS)

[USER] node / node-22 (complément – voir §10 pour la base)

DESCRIPTION : Runtime JavaScript V8 – exécute JavaScript côté serveur.

OPTIONS SUPPLÉMENTAIRES :

-e 'CODE' / --eval 'CODE'	Exécute du code JS directement
-p 'CODE' / --print 'CODE'	Exécute et affiche le résultat
-r MODULE / --require MODULE	Charge un module avant exécution
--input-type=TYPE	Type d'entrée (module, commonjs)
--inspect[=HOST:PORT]	Active le débogueur DevTools
--inspect-brk[=HOST:PORT]	Débogueur avec pause au démarrage
--max-old-space-size=N	Limite mémoire heap en Mo
--experimental-vm-modules	VM modules (ESM)
--no-deprecation	Supprime les avertissements de déprecation
--pending-deprecation	Active les déprecations en attente
--trace-warnings	Trace les avertissements avec stack trace
--version / -v	Version de Node.js
--env-file=FICHER	Charge un fichier .env
--watch	Redémarre si les fichiers changent
--watch-path=CHEMIN	Surveille ce chemin

ONE-LINERS UTILES :

```

$ node -e "console.log(process.versions)" # Versions de Node et ses dépendances
$ node -e "const fs = require('fs'); console.log(fs.readdirSync('.'))"
$ node -p "JSON.stringify(JSON.parse(require('fs').readFileSync('/dev/stdin','utf8')), null, 2)" # Format JSON
$ node --inspect mon_script.js           # Débogue via Chrome DevTools

```

[USER] npm / npx (complément – voir §10 pour la base)

DESCRIPTION : npm = Node Package Manager | npx = exécuteur de paquets npm

COMMANDES SUPPLÉMENTAIRES :

npm audit	Audit de sécurité des dépendances
npm audit fix	Corrige automatiquement les vulnérabilités
npm audit fix --force	Force la correction (peut changer les APIs)
npm outdated	Liste les paquets avec mise à jour disponible
npm update	Met à jour les dépendances
npm update PAQUET	Met à jour un paquet spécifique
npm ci	Install propre depuis package-lock.json (CI/CD)
npm pack	Crée une archive .tgz du paquet
npm publish	Publie le paquet sur npmjs.com
npm link	Lie le paquet local pour développement
npm link NOM	Utilise un paquet lié localement
npm exec COMMANDE	Exécute une commande du bin du paquet
npm set-script NOM CMD	Ajoute un script npm dans package.json
npm fund	Affiche les infos de financement des dépendances
npm ls --depth=0	Dépendances directes seulement
npm ls --prod	Seulement les dépendances de production
npm ls --dev	Seulement les dépendances de développement
npm config set registry URL	Change le registre (ex: miroir interne)
npm config get prefix	Répertoire d'installation des paquets globaux
npm cache clean --force	Vide le cache npm
npm doctor	Diagnostic de l'environnement npm
npm diff [PAQUET]	Diff entre version installée et publiée
npm explain PAQUET	Pourquoi ce paquet est installé
npm dedupe	Déduplique les dépendances
npm shrinkwrap	Génère npm-shrinkwrap.json (équivalent package-lock)

NPX SPÉCIFICITÉS :

npx PAQUET	Exécute sans installation permanente
npx -y PAQUET	Pas de confirmation d'installation
npx --no-install PAQUET	N'installe pas, échoue si absent
npx PAQUET@VERSION	Version spécifique
npx --package P1 --package P2 CMD	Exécute CMD avec plusieurs paquets disponibles

```

EXEMPLES      :
$ npm ci           # Install reproductible (CI/CD)
$ npm audit        # Rapport de sécurité
$ npm outdated     # Packages à mettre à jour
$ npx create-react-app mon-app # Crée un projet React sans install globale
$ npx prettier --write "**/*.js" # Formate le code JS
$ npm ls --depth=0 # Dépendances directes

```

G. PROFILING & COUVERTURE DE CODE

```
[USER] gprof (complément – voir §10 pour la base)
```

DESCRIPTION : GNU Profiler – analyse les données de profiling générées par un programme compilé avec -pg et génère un rapport de performance.

POURQUOI : Identifier les fonctions qui consomment le plus de temps CPU pour optimiser le code aux bons endroits.

PRÉREQUIS : Compiler avec gcc -pg (instrumente le code pour le profiling).

OPTIONS :

- b / --brief Sans description des champs (sortie courte)
- p / --flat-profile[=SYMB] Profil plat (temps par fonction)
- q / --call-graph[=SYMB] Graphe d'appels
- A / --annotated-source[=SYMB] Source annotée avec comptes
- J / --no-annotated-source Pas de source annotée
- l / --line Profil ligne par ligne
- c / --static-call-graph Inclut les arcs non exécutés
- i / --file-info Infos sur gmon.out
- s / --sum Additionne plusieurs gmon.out
- e FONCTION Exclut cette fonction du graphe
- E FONCTION Exclut cette fonction du graphe ET de ses parents
- f FONCTION Affiche seulement cette fonction
- F FONCTION Seulement cette fonction et ses enfants
- n N / --number-of-nodes=N Limite les nœuds du graphe
- w N / --width=N Largeur de sortie
- D / --ignore-zeros Ignore les fonctions à 0 appels

EXÉCUTABLE : Programme profilé

FICHIER_GMON : Données de profiling (défaut: gmon.out)

FLUX DE TRAVAIL :

1. Compiler avec -pg : gcc -pg -o prog prog.c
2. Exécuter : ./prog (génère gmon.out)
3. Analyser : gprof prog gmon.out | less

EXEMPLES :

```

$ gcc -pg -O2 -o mon_prog mon_prog.c # Compile avec instrumentation
$ ./mon_prog                          # Exécute (crée gmon.out)
$ gprof mon_prog gmon.out              # Rapport complet
$ gprof -b mon_prog gmon.out           # Rapport sans descriptions
$ gprof -q -b mon_prog gmon.out        # Graphe d'appels seulement
$ gprof -p -b mon_prog gmon.out        # Profil plat seulement
$ gprof -b mon_prog gmon.out | head -40 # Top des fonctions les plus lentes

```

H. PATCH & FORMATAGE DE CODE

```
[USER] patch
```

DESCRIPTION : Applique des fichiers de patch (diff unifié ou contextuel) à des fichiers source. L'outil standard pour distribuer et appliquer des modifications de code.

POURQUOI : Appliquer des correctifs de code (patches noyau Linux, patches de sécurité, modifications de mainteneurs) sans remplacer entièrement les fichiers.

CONTEXTE : Développement logiciel, maintenance de paquets, backporting de fixes.

OPTIONS :

- p N / --strip=N Supprime N niveaux du chemin (p0=chemin exact, p1=supprime le 1er /, très courant pour les patches git)
- R / --reverse Applique le patch à l'envers (désappliquer)
- N / --forward Ignore les patches déjà appliqués
- E / --remove-empty-files Supprime les fichiers vides après patch
- b / --backup Crée des backups (.orig)
- backup-if-mismatch Backup seulement si décalage
- suffix=SUFFIXE Suffixe des fichiers backup (défaut: .orig)

```

-d RÉPERTOIRE           Applique dans ce répertoire
-i FICHIER / --input=FICHIER Fichier de patch (défaut: stdin)
--dry-run               Simulation (ne modifie rien)
-f / --force            Force (ignore les erreurs)
-F N / --fuzz=N         Niveau de tolérance au décalage de contexte (0-3)
-s / --silent           Mode silencieux
-v / --verbose          Verbeux
-o FICHIER              Écrit la sortie dans ce fichier (pas en place)
--no-backup-if-mismatch Pas de backup si décalage
-r FICHIER / --reject-file=FICHIER Fichier pour les rejets (défaut: .rej)
--merge / --merge=merge Merge les conflits style git (3-way)

```

```

EXEMPLES :
$ patch -p1 < correction.patch      # Applique un patch git standard
$ patch -p0 < fix.diff              # Chemin exact dans le diff
$ patch -p1 --dry-run < patch.diff  # Teste sans modifier
$ patch -p1 -R < patch.diff         # Désapplique le patch
$ patch -p1 -b < patch.diff         # Avec backup des fichiers originaux
$ patch -p1 -d /usr/src/linux < kpatch.diff # Dans un répertoire spécifique
# Créer un patch avec diff :
$ diff -u original.c modifié.c > fix.patch
$ diff -ruN original_dir/ modifié_dir/ > changes.patch

```

```
[USER] indent
```

```

DESCRIPTION : Formate et indente automatiquement du code source C selon différents
              styles (K&R, GNU, BSD, Linux, etc.).
POURQUOI    : Standardiser le style de code C, convertir entre styles, préparer
              du code pour la revue, enforcer les conventions d'un projet.
CONTEXTE    : Développement C, maintenance de code legacy, revue de code.
OPTIONS      :
-gnu          Style GNU (défaut sur les systèmes GNU)
-kr / -K&R    Style Kernighan & Ritchie
-orig         Style original Berkeley
-linux        Style noyau Linux (K&R + tab=8)
-nbad / -bad  Sans/avec ligne vide après déclarations
-bap / -nbap  Avec/sans ligne vide après procédures
-bbb / -nbbs  Ligne vide avant bloc commentaire en gras
-bc / -nbc    Retour ligne après virgule dans déclarations
-bl / -br     Accolade gauche : nouvelle ligne / même ligne
-bli N        Indentation des accolades gauches
-bs / -nbs    Espace avant sizeof
-c N          Colonne pour commentaires (défaut: 33)
-cd N         Colonne pour commentaires de déclarations
-cn N         Colonne pour commentaires non-locaux
-cp N         Colonne pour commentaires de directives préprocesseur
-cs / -ncs    Espace/pas d'espace après cast
-d N          Niveau d'indentation des commentaires
-di N         Colonne pour les déclarations de variables
-dj / -ndj    Justification gauche/droite des déclarations
-ei / -nei     Indentation des else-if
-fca / -nfca  Formate/ne formate pas les commentaires
-i N          Taille de l'indentation (défaut: 2 pour GNU, 8 pour Linux)
-il N         Taille indentation du premier niveau
-l N          Longueur de ligne max (défaut: 78)
-lp / -nlp    Aligne les parenthèses
-npcs / -pcs  Espace/pas d'espace avant appels de fonctions
-psl / -npsl  Nom de fonction sur nouvelle ligne (prototype)
-sc / -nsc    Commentaires multi-lignes avec * au début
-sob / -nsob  Supprime les lignes vides superflues
-st           Sortie sur stdout
-T NOM        NOM est un typedef (pas de parenthèses autour)
-v           Verbeux
-o FICHIER    Fichier de sortie
FICHIER       Fichier source à formater (modifié en place !)
EXEMPLES      :
$ indent -linux -i8 mon_code.c      # Style noyau Linux
$ indent -kr mon_code.c             # Style K&R
$ indent -gnu mon_code.c            # Style GNU
$ indent -st mon_code.c             # Affiche sur stdout (sans modifier)
$ indent -kr -i4 -l100 mon_code.c   # K&R, indent 4, max 100 cols
$ indent -orig -o formaté.C original.c # Vers un nouveau fichier

```

```
[USER] gcov
```

```
DESCRIPTION : Outil de couverture de code pour GCC – affiche quelles lignes de code
              ont été exécutées et combien de fois pendant les tests.
POURQUOI    : Mesurer l'efficacité des tests unitaires, identifier le code mort,
              s'assurer que les cas limites sont testés.
PRÉREQUIS   : Compiler avec gcc --coverage (ou -fprofile-arcs -ftest-coverage).
OPTIONS      :
  -a / --all-blocks      Compte chaque bloc (pas seulement les lignes)
  -b / --branch-probabilities Affiche les stats de branches (if/else)
  -c / --branch-counts   Compte des branches (pas de probabilités)
  -d / --display-progress Affiche la progression
  -f / --function-summaries Résumé par fonction
  -i / --json-format      Sortie JSON (gcov ≥ 9)
  -j / --human-readable  Tailles lisibles par l'humain
  -l / --long-file-names Noms de fichiers longs (include)
  -m / --demangled-names Démangle les noms C++
  -n / --no-output       Pas de fichiers .gcov (stats seulement)
  -o RÉPERTOIRE          Répertoire des fichiers .gcda/.gcno
  -p / --preserve-paths  Préserve les chemins dans les noms de fichiers
  -q / --use-colors      Couleurs (rouge=non couvert, bleu=couvert)
  -r / --relative-only   Seulement les fichiers du répertoire courant
  -s PRÉFIXE             Préfixe source à supprimer
  -t / --stdout          Sortie sur stdout
  -u / --unconditional-branches Inclut les branches inconditionnelles
  -x / --hash-filenames  Hash dans les noms de fichiers
FICHIER.c             Fichier source à analyser
```

FLUX DE TRAVAIL :

1. Compiler avec coverage : gcc --coverage -o prog prog.c
2. Exécuter : ./prog (génère prog.gcda et prog.gcno)
3. Analyser : gcov prog.c (génère prog.c.gcov)

FORMAT DU FICHIER .gcov :

```
##### : Ligne jamais exécutée (non couverte)
N       : Ligne exécutée N fois
-       : Ligne non exécutable (déclaration, commentaire)
===== : Bloc non exécuté dans branche
```

EXEMPLES :

```
$ gcc --coverage -o mon_prog mon_prog.c # Compile avec coverage
$ ./mon_prog                             # Exécute les tests
$ gcov mon_prog.c                         # Génère mon_prog.c.gcov
$ gcov -b mon_prog.c                     # Avec stats de branches
$ gcov -f mon_prog.c                     # Résumé par fonction
$ cat mon_prog.c.gcov | grep "#####"    # Lignes non couvertes
# Avec lcov pour un rapport HTML :
$ lcov --capture --directory . --output-file cov.info
$ genhtml cov.info --output-directory cov_html/
```

J. AIDE-MÉMOIRE : DRAPEAUX GCC ESSENTIELS

COMPILATION STANDARD :

```
gcc -Wall -Wextra -std=c11 -O2 -o prog prog.c    # C11, optimisé, warnings
g++ -Wall -Wextra -std=c++17 -O2 -o prog prog.cpp # C++17
gcc -g -O0 -fsanitize=address,undefined -o prog prog.c # Debug + sanitizers
```

OPTIMISATION :

```
-O0      Sans optimisation (débogage)
-O1      Optimisation de base
-O2      Optimisation recommandée (production)
-O3      Optimisation agressive (peut changer le comportement)
-Os      Optimise pour la taille (embarqué)
-Oz      Taille minimale absolue (clang)
-Og      Optimisé pour le débogage
-march=native Optimise pour le CPU local (non-portable)
-flto    Link-Time Optimization (encore plus rapide)
```

DÉBOGAGE & ANALYSE :

```
-g / -g3      Infos de débogage DWARF
-ggdb3        Infos DWARF étendues pour GDB
-fsanitize=address AddressSanitizer (buffer overflow, use-after-free)
-fsanitize=undefined UndefinedBehaviorSanitizer
```

-fsanitize=thread	ThreadSanitizer (data races)
-fsanitize=memory	MemorySanitizer (valeurs non initialisées, clang)
-fsanitize=leak	LeakSanitizer (fuites mémoire)
-fno-omit-frame-pointer	Garde le frame pointer (meilleurs backtraces)
-pg	Instrumentation gprof
--coverage	Instrumentation gcov
 WARNINGS (toujours activer en dev) :	
-Wall	Warnings essentiels
-Wextra	Warnings supplémentaires
-Wpedantic / -pedantic	Strictement standard
-Werror	Traite les warnings comme erreurs
-Wshadow	Variable cache une autre
-Wformat-security	Problèmes de format string
-Wconversion	Conversions implicites
-Wnull-dereference	Déréférencement de NULL (GCC ≥ 6)
-fanalyzer	Analyse statique intégrée (GCC ≥ 10)
 SÉCURITÉ (production) :	
-D_FORTIFY_SOURCE=2	Protection des fonctions de bibliothèque
-fstack-protector-strong	Protection contre débordements de pile
-fPIE / -pie	Position-Independent Executable (ASLR)
-fPIC	Position-Independent Code (bibliothèques)
-Wl,-z,relro,-z,now	RELRO + liaison immédiate (ld flags)
 PRÉPROCESSEUR :	
-DNOM=VALEUR	Définit une macro
-UNOM	Annule une macro
-E	Seulement le préprocesseur (sortie stdout)
-M / -MM	Génère les dépendances (pour Makefile)
-MF FICHIER	Écrit les dépendances dans ce fichier
-I RÉPERTOIRE	Chemin d'en-têtes supplémentaire
-isystem RÉPERTOIRE	Comme -I mais sans warnings
 LINKAGE :	
-l NOM	Lie avec libNOM.so
-L RÉPERTOIRE	Cherche les bibliothèques ici
-Wl,OPTION	Passe OPTION au linker
-Wl,-rpath,RÉPERTOIRE	Chemin runtime pour les .so
-static	Linkage entièrement statique
-shared	Crée une bibliothèque partagée
-soname NOM	SONAME de la bibliothèque partagée
-Wl,--as-needed	Ne lie que les bibliothèques réellement utilisées